

EVA System Architecture

Общее описание системы

EVA — это самопознающая когнитивная система с механизмом самообучения. Система построена по принципу модульной архитектуры, где каждый компонент выполняет специализированную задачу и взаимодействует с другими через централизованные шины событий и команд.

Ключевые принципы:

- **Самопознание** — система отслеживает свои знания, выявляет пробелы и противоречия
 - **Ленивая загрузка** — модели загружаются только при первом запросе, экономя ресурсы
 - **Приоритизация** — критические задачи (ответы пользователю) имеют высший приоритет
 - **Асинхронность** — фоновые задачи (майнинг концептов, анализ противоречий) выполняются в отдельных потоках
 - **Координация** — единая точка входа для доступа к модели предотвращает конфликты
-

1. Высокоуровневая архитектура

1.1 Слои системы

Система состоит из четырёх логических слоёв:

Слой представления (Web UI)

Веб-интерфейс на базе Flask обеспечивает взаимодействие с пользователем. Браузерный клиент на JavaScript отправляет запросы через AJAX и получает потоковые ответы через Server-Sent Events (SSE). Интерфейс поддерживает сессии, историю диалогов и отображение статистики обучения.

Слой координации (CoreBrain)

Центральный координатор, управляющий всеми компонентами системы. CoreBrain наследует функциональность от девяти миксинов, каждый из которых отвечает за свою область: конфигурация, компоненты, запросы, мониторинг, память, состояние, события, команды и отслеживание процессов.

Слой обработки знаний

Компоненты этого слоя отвечают за извлечение, хранение и анализ знаний. Сюда входят системы концептов и противоречий, граф памяти FractalGraph v2, куратор графа и Википедия KB.

Слой генерации

Отвечает за генерацию текста с использованием языковых моделей. Включает универсальный

генератор с поддержкой OpenVINO, менеджер доступа к модели и систему отложенных команд.

2. Пользовательский интерфейс

2.1 Веб-интерфейс

Flask-сервер обслуживает веб-интерфейс и API endpoints. Основная страница загружает JavaScript-клиент, который управляет взаимодействием с пользователем.

Основные возможности интерфейса:

- Отправка сообщений и получение потоковых ответов
- Управление сессиями диалога
- Просмотр статистики обучения системы
- Визуализация графа знаний
- Загрузка документов для анализа

2.2 Потокковая передача ответов

При отправке сообщения пользователем интерфейс открывает XMLHttpRequest соединение с endpoint `/api/chat/stream`. Сервер возвращает данные в формате Server-Sent Events (SSE), позволяя передавать текст пользователю по мере его генерации.

JavaScript-клиент накапливает полученные фрагменты и отображает их в реальном времени. При получении события `complete` с полным очищенным текстом клиент заменяет накопленные фрагменты финальным результатом.

3. Обработка пользовательского запроса

3.1 Входная точка

Когда запрос пользователя поступает в систему, он проходит через несколько этапов обработки.

Предобработка включает извлечение сущностей и определение намерений пользователя. На этом этапе система также проверяет запрос на соответствие этическим нормам.

Построение контекста — система собирает релевантную информацию из нескольких источников: концепты, связанные с запросом, противоречия, обнаруженные ранее, и разрешённые знания из кэша. Эта информация дополняет оригинальный запрос перед отправкой модели.

3.2 Координация доступа к модели

ModelAccessManager — единственная точка входа для генерации текста. Прежде чем запрос поступит к модели, он проходит через менеджер доступа, который:

- Принимает запрос и присваивает ему приоритет (CRITICAL для запросов пользователя)
- Проверяет, не занят ли генератор другим запросом
- Если занят, запрос ожидает в очереди с учётом приоритета
- Когда модель освобождается, запрос выполняется
- После завершения публикуется событие о завершении

Такая координация предотвращает конфликты, когда несколько компонентов одновременно пытаются использовать модель.

3.3 Генерация ответа

UnifiedGenerator выбирает оптимальную модель для запроса. Система использует две модели:

- **LOGIC/CONTEXT (CPU)** — для логических рассуждений и работы с контекстом. Работает в режиме минимальной задержки (LATENCY), оптимизирована для быстрых ответов.
- **CODER (GPU)** — для генерации кода и самодialogов. Работает в режиме максимальной пропускной способности (THROUGHPUT), эффективно обрабатывает длинные последовательности.

При первом запросе модель загружается в память и остаётся там для последующих запросов (ленивая загрузка).

3.4 Постобработка и сохранение

После генерации ответа система выполняет несколько действий:

- Извлекает концепты из запроса и ответа
 - Сохраняет концепты в граф памяти
 - Генерирует потенциальные противоречия для новых концептов
 - Сохраняет опыт взаимодействия в FractalGraph v2
-

4. Система концептов

4.1 Двухуровневая архитектура

Система концептов состоит из двух уровней, различающихся по скорости и глубине анализа.

Быстрый уровень (ConceptExtractor) работает синхронно при каждом запросе. Он анализирует текст запроса и ответа, выделяет ключевые термины на основе частотного анализа и создаёт концепты с базовыми фактами (отношения типа является, имеет свойство, может, связано с).

Этот уровень не выполняет валидацию — концепты сохраняются сразу в граф и доступны для использования в следующих запросах.

Глубокий уровень (ConceptMiner) работает асинхронно в фоновом режиме. Он анализирует кластеры в графе памяти, выявляя семантические лакуны — области, где модели недостаточно информации для связного понимания.

Математически лакуна определяется как минимальное расстояние от центроида кластера до его членов. Если это расстояние превышает адаптивный порог, создаётся кандидат на новый концепт.

4.2 Жизненный цикл концепта

Концепты проходят через несколько стадий жизненного цикла:

- **PROVISIONAL** — концепт только обнаружен, требует валидации
- **CONFIRMED** — концепт прошёл проверку (NLI, онтология, этика, верификация через веб)
- **STABLE** — концепт подтверждён и не имеет противоречий после нескольких циклов
- **ARCHIVED** — концепт отклонён или имеет низкую достоверность

4.3 Взаимодействие с самодialogом

Когда концепт подтверждён или обнаружен, он добавляется в очередь самодialogа. Система инициирует исследовательский диалог, состоящий из четырёх этапов:

1. **ASSISTANT** представляет концепт и даёт определение
2. **CRITIC** выявляет проблемы и противоречия в определении
3. **LEARNER** предлагает направления для углублённого изучения
4. **TEACHER** формулирует практические рекомендации

Результаты диалога сохраняются в кэш и используются для обогащения будущих ответов.

5. Система противоречий

5.1 Два подхода к обнаружению

Генератор противоречий (ContradictionGenerator) работает на основе шаблонов. Для каждого концепта система создаёт противоположные точки зрения, используя заранее определённые шаблоны по доменам (общий, технологии, наука, философия).

Это быстрый метод, который гарантированно генерирует противоречие для обсуждения, даже если реального конфликта в знаниях нет.

Майнер противоречий (ContradictionMiner) анализирует структуру графа памяти для обнаружения реальных логических противоречий. Для каждой пары узлов вычисляется:

- Семантическое сходство (косинусное расстояние между эмбедингами)
- Степень противоречия через NLI-модель (логический анализ)

Если сходство превышает 0.75 и противоречие превышает 0.65, пара становится кандидатом.

5.2 Кластеризация и формулировка

Кандидаты объединяются в кластеры через транзитивное замыкание. Если узел A противоречит B, а B противоречит C, то все три формируют один кластер противоречий.

Для каждого кластера система генерирует формулировку: название противоречия, описание, вопрос для разрешения. Это делается через языковую модель с низкой температурой для воспроизводимости.

5.3 Разрешение противоречий

Обнаруженные противоречия добавляются в очередь самодialogа. Система проводит четырёхэтапный диалог:

1. **ASSISTANT** представляет обе точки зрения
2. **CRITIC** анализирует сильные и слабые стороны каждой
3. **LEARNER** ищет синтез или компромисс
4. **TEACHER** формулирует финальное разрешение

Результат сохраняется как разрешённое противоречие и может использоваться в будущих ответах.

6. Самодialog и обучение

6.1 Назначение самодialogа

Самодialog — это механизм самообучения, при котором система ведёт внутренний диалог для исследования концептов, разрешения противоречий и выявления пробелов в знаниях.

В отличие от простого сохранения фактов, самодialog позволяет системе:

- Глубже понять концепт через критический анализ
- Обнаружить неявные противоречия
- Сгенерировать новые вопросы для исследования
- Сформировать практические рекомендации

6.2 Приоритизация задач

Система использует три очереди с приоритетами:

1. **Противоречия** — высший приоритет, требуют разрешения для согласованности знаний
2. **Концепты** — средний приоритет, исследуются для углубления понимания
3. **История разговоров** — низкий приоритет, анализируется для выявления новых тем

Решения о приоритетах принимаются на основе оценки важности и срочности задачи.

6.3 Структура диалогов

Каждый диалог состоит из последовательных реплик от имени различных ролей:

- **ASSISTANT** — нейтральное представление темы
- **CRITIC** — критический анализ с выявлением проблем
- **LEARNER** — предложение новых направлений исследования
- **TEACHER** — итоговые рекомендации и выводы

После завершения диалога результаты сохраняются в кэш и граф памяти.

7. Шина событий (EventBus)

7.1 Назначение и принцип работы

EventBus — это централизованная шина для коммуникации между компонентами через события. Вместо прямой связи компоненты публикуют события и подписываются на интересующие их типы.

Это обеспечивает слабую связанность — компоненты не знают о существовании друг друга, но могут реагировать на события системы.

7.2 Типы событий

Системные события сигнализируют о состоянии системы: запуск, остановка, готовность, ошибки.

События конвейера отражают процесс генерации: начало, прогресс, завершение, ошибка.

События памяти уведомляют об изменениях в графе: обновление узлов, завершение кластеризации.

События обучения информируют о действиях самообучающейся системы: обнаружение концепта, противоречия, завершение диалога.

События модели отражают доступ к модели генерации: запрос, завершение, ошибка.

7.3 Приоритеты и подписки

События имеют приоритеты (LOW, NORMAL, HIGH, CRITICAL). Подписчики получают события в порядке приоритета. Можно указать конкретный обработчик и приоритет подписки.

8. Система отложенных команд

8.1 Архитектура очередей

DeferredCommandSystem управляет асинхронным выполнением команд с учётом приоритетов. Вместо немедленного выполнения команды ставятся в очередь и

обрабатываются пулом потоков.

CRITICAL — немедленные операции, например генерация ответа пользователю

HIGH — срочное обучение, майнинг концептов и противоречий

NORMAL — регулярные операции, обработка диалогов

LOW — фоновые оптимизации, например сжатие контекста

8.2 Защита от перегрузки

Система отслеживает нагрузку на CPU и размер очередей. При превышении пороговых значений низкоприоритетные команды сбрасываются для освобождения ресурсов.

8.3 Восстановление после сбоев

Если команда завершается с ошибкой, система автоматически планирует повторную попытку. После нескольких неудачных попыток команда помечается как Failed и может быть перезапущена вручную.

9. OpenVINO и оптимизация

9.1 Реестр генераторов

OpenVINOGeneratorRegistry — это синглтон, обеспечивающий шаринг модели между компонентами. Когда нескольким генераторам нужна модель на одном устройстве, они получают ссылку на существующий экземпляр вместо создания нового.

Это критически важно для GPU, где видеопамять ограничена. Вместо загрузки нескольких копий модели система загружает одну и использует её совместно.

9.2 Ленивая загрузка

При создании OpenVINOGenerator модель не загружается сразу. Вместо этого запоминаются параметры конфигурации. Модель загружается при первом вызове генерации.

Это ускоряет старт системы и снижает нагрузку на SSD, так как не все модели загружаются при каждом запуске.

9.3 Конфигурация устройств

CPU (Logic/Context):

- Режим LATENCY для минимальной задержки
- NUM_STREAMS=AUTO для использования всех ядер
- Оптимизированный планировщик для параллельных запросов

GPU (Coder/Self-dialog):

- Режим THROUGHPUT для максимальной пропускной способности
 - Кэш KV для эффективного переиспользования вычислений
 - Prefix caching для общих префиксов в промптах
-

10. Инициализация системы

10.1 Порядок запуска

При старте CoreBrain инициализирует компоненты в определённом порядке:

1. Создаётся EventBus как централизованная шина
2. Инициализируется DeferredCommandSystem с привязкой к EventBus
3. Создаются менеджеры (ресурсов, конфигурации, памяти)
4. Инициализируется универсальный генератор с OpenVINO (модели не загружаются)
5. Запускаются фоновые компоненты: ConceptMiner, ContradictionMiner, SelfDialogLearning

10.2 Подписки на события

После создания компонентов каждый подписывается на релевантные события EventBus. ConceptMiner слушает события памяти и готовности системы. ContradictionMiner реагирует на создание узлов. SelfDialogLearning подписан на события концептов и противоречий.

10.3 Пост-инициализация

После базовой инициализации запускаются дополнительные сервисы: FractalGraph v2, GraphCurator для оптимизации графа. Система переходит в рабочее состояние и готова принимать запросы.

11. Веб-API

11.1 Основные endpoints

/api/chat — классический чат с полным ответом в JSON

/api/chat/stream — потоковая передача ответа через SSE

/api/sessions — управление сессиями пользователя

/api/status — состояние системы и компонентов

/api/knowledge-graph — данные графа знаний (узлы, связи)

/api/analytics — статистика обучения и метрики

11.2 Формат потокового ответа

Сервер отправляет события в формате SSE:

- `start` — начало генерации
 - `chunk` — фрагмент текста
 - `complete` — полный очищенный текст
 - `done` — завершение с финальными метаданными
 - `error` — сообщение об ошибке
-

12. Компоненты и их ответственность

Компонент	Файл	Ответственность
CoreBrain	core/core_brain.py	Центральная координация
EventBus	core/event_bus.py	Коммуникация через события
DeferredCommandSystem	core/deferred_command_system.py	Асинхронные команды
ModelAccessManager	core/model_access_manager.py	Координация доступа к модели
UnifiedGenerator	core/unified_generator.py	Генерация текста
OpenVINOGenerator	core/opencvino_generator.py	OpenVINO обёртка
ConceptExtractor	knowledge/concept_extractor.py	Быстрое извлечение концептов
ConceptMiner	knowledge/concept_miner.py	Глубокий анализ кластеров
ContradictionGenerator	contradiction/contradiction_generator.py	Генерация шаблонных противоречий
ContradictionMiner	contradiction/contradiction_miner.py	Аналитическое обнаружение противоречий
SelfDialogLearning	learning/dialog_core.py	Самодialog и обучение
FractalGraphV2	memory/fractal_graph_v2/	Хранение знаний
GraphCurator	knowledge/graph_curator.py	Оптимизация графа

13. Конфигурация

Система настраивается через `brain_config.json`. Ключевые секции:

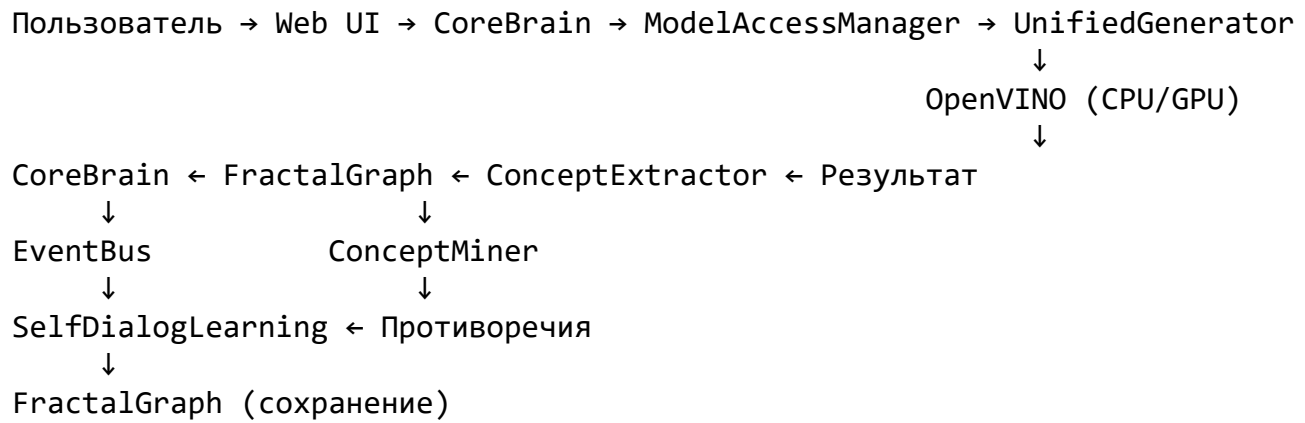
model — выбор модели, устройства (CPU/GPU), пути к файлам моделей

concept_system — включение экстрактора и майнера, интервалы проверки, пороги

contradiction_system — аналогично для системы противоречий

self_dialog_learning — интервалы автодиалога, пороги качества

14. Поток данных (сводка)



Документ подготовлен для презентации архитектуры системы EVA

Обновлено: April 14, 2026